CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 5

Outline

HTML APIs

1. What is an API?
2. Drag and Drop
3. GeoLocation
4. Storage

What is an API

HTML APIs

1. What is an API?
2. Drag and Drop
3. GeoLocation
4. Storage

What is an API?

- API: **A**pplication **P**rogramming **I**nterface
- APIs just allow applications to communicate with one another.

API examples

- **The Java API**
  - Software components available “out of the box” to anyone who has installed the Java Platform
  - Some examples:
    - JDBC: Java Database Connectivity
    - JPA: Java Persistence
    - JSF: JavaServer Faces
    - JAX-RS: Java API for RESTful Web Services

API examples

- **Google APIs**
  - allow you to connect your code to the whole range of Google services, from Maps to Translate
- **Facebook APIs**
  - allow you to programmatically access Facebook's social graph and marketing tools
- **Twitter APIs**
  - web-based JSON API that allows developers to programmatically interact with Twitter data

Some of HTML5 APIs

Api Name	Description
Drag and Drop	Supports dragging and dropping items within and between browser
File API	Provides programs with secure access to the device's file system.
Forms	Gives programs access to the new data types defined in HTML5.
Geolocation	Provides web applications with access to geographical location data about the user's device.
History API	Allows programs to manipulate the browser history.
Indexed database	Creates a simple client-side database system in the web browser.
Server-sent events	Allows the server to push data to the browser without the browser needing to request it.
Web storage	Allows the storage of key-value pairs in the browser.
Web sockets	Opens an interactive communication session between the browser and server.
Web Workers	Allows JavaScript to execute scripts in the background.

Drag and Drop API

HTML APIs

1. What is an API?
2. Drag and Drop
3. GeoLocation
4. Storage

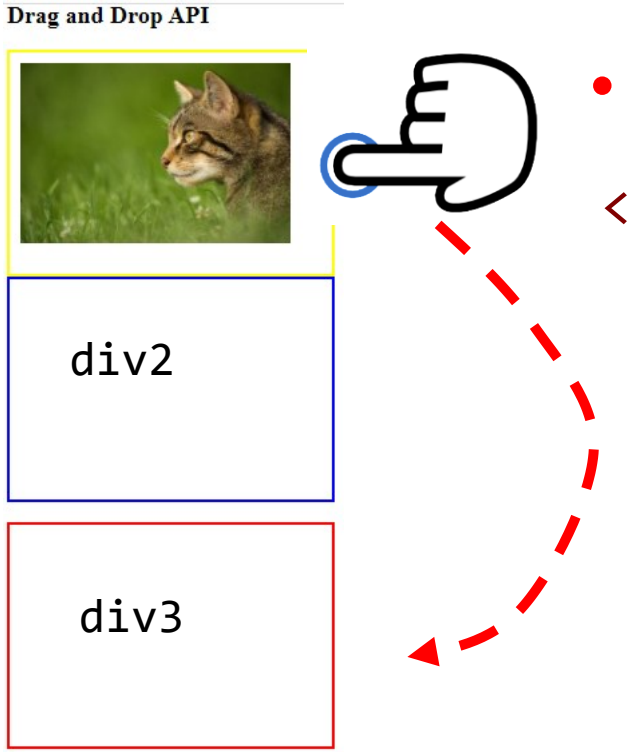
Drag and Drop

- Allows the user to drag and drop an element to another location.
- Part of the HTML5 standard, and any element can be draggable.
- The drop location may be a different application.

Drap & Drop Events

Event	On Event Handler	Fires when…
drag	ondrag	…a *dragged item* (element or text selection) is dragged.
dragend	ondragend	…a drag operation ends (such as releasing a mouse button or hitting the Esc key; see Finishing a Drag.)
dragenter	ondragenter	…a dragged item enters a valid drop target. (See Specifying Drop Targets.)
dragexit	ondragexit	…an element is no longer the drag operation's immediate selection target.
dragleave	ondragleave	…a dragged item leaves a valid drop target.
dragover	ondragover	…a dragged item is being dragged over a valid drop target, every few hundred milliseconds.
dragstart	ondragstart	…the user starts dragging an item. (See Starting a Drag Operation.)
drop	ondrop	…an item is dropped on a valid drop target. (See Performing a Drop.)

Steps for Drag&Drop(1/4)



- Step 1: Make object draggable

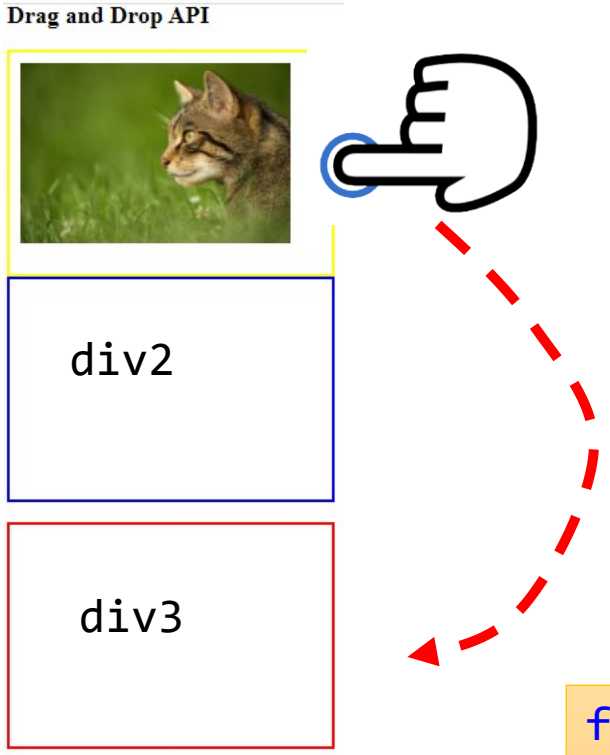
```
<img id="cat" src="cat.jpg" draggable="true">
```

DOM Tree

```
•<html lang="en"> == $0
  ▶ <head>…</head>
  ▼ <body>
      <h1>Week-3 Examples</h1>
      <h2>Drag and Drop API</h2>
      ▼ <div id="div1" ondrop="drop(event)" ondragover="allowDrop(event)">
          <img id="cat" src="cat.jpg" draggable="true" ondragstart="drag(event)">
      </div>
      <div id="div2" ondrop="drop(event)" ondragover="allowDrop(event)">
          </div>
      <div id="div3" ondrop="drop(event)" ondragover="allowDrop(event)">
          </div>
    </body>
  </html>
```

See: html5-api.html

Steps for Drag&Drop(2/4)



- Step 2: Specify what should happen when the element is dragged

```
<img id="cat" src="cat.jpg" draggable="true"
                ondragstart="drag(event)">
```

calls function

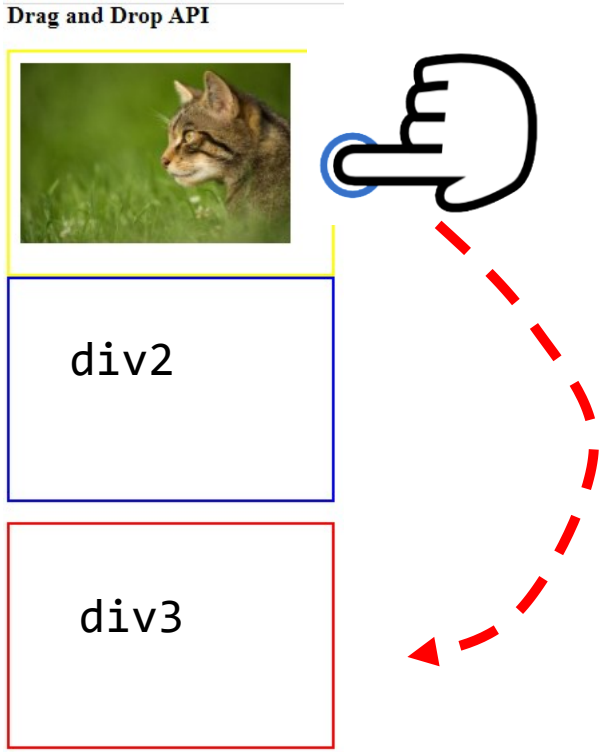
id of the draggable element ("cat")

```
function drag(event) {
    event.dataTransfer.setData("text/plain", event.target.id);
}
```

property that holds the event's data

Datatype & Data

Steps for Drag&Drop(3/4)



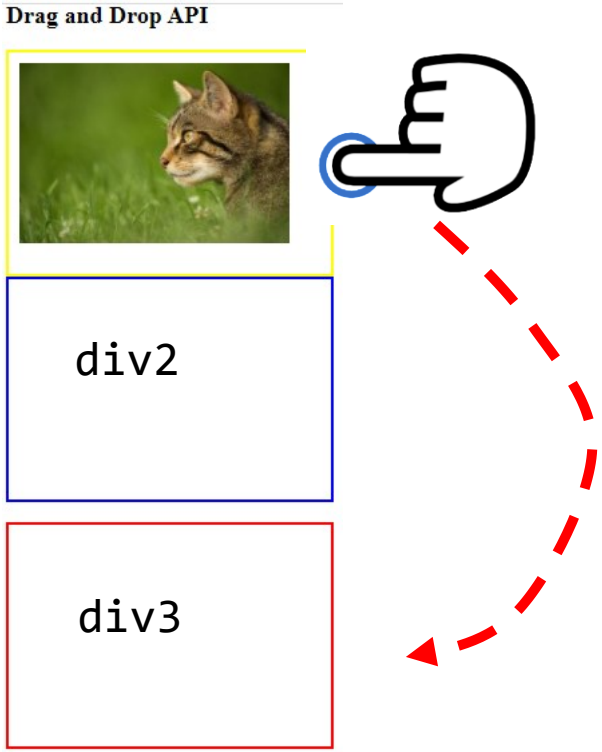
- Step 3: specify where the dragged data will be dropped

```
<div id="div3" ondragover="allowDrop(event)">
</div>
```

```
function allowDrop(event) {
      event.preventDefault();
}
```

By default, data/elements cannot be dropped in other elements. To allow a drop, we must prevent the default handling of the element.

Steps for Drag&Drop(4/4)



- Step 4: do the drop

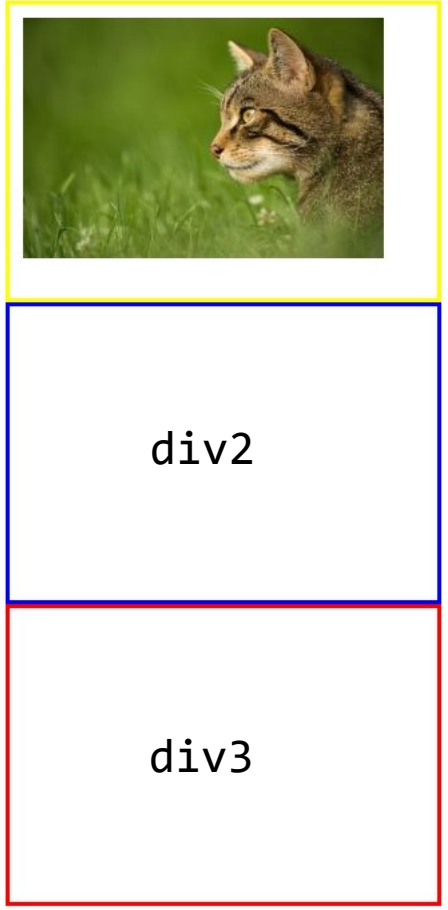
```
<div id="div3" ondrop="drop(event)"
     ondragover="allowDrop(event)">
</div>
```

```
function drop(event) {
     event.preventDefault();
     var data = event.dataTransfer.getData("text/plain");
     event.target.appendChild(document.getElementById(data));
}
```

When the dragged data is dropped, a drop event occurs.

After dropping

Drag and Drop API



DOM Tree

```
<html lang="en">
▶ <head>…</head>
▼ <body>
    <h1>Week-3 Examples</h1>
    <h2>Drag and Drop API</h2>
    <div id="div1" ondrop="drop(event)" ondragover="allowDrop(event)"></div>
    <div id="div2" ondrop="drop(event)" ondragover="allowDrop(event)">
      </div>
  ▼ <div id="div3" ondrop="drop(event)" ondragover="allowDrop(event)">
      <img id="cat" src="cat.jpg" draggable="true" ondragstart="drag(event)">
    </div>
  </body>
</html>
```

For More Info

- https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API
- https://www.w3schools.com/html/html5_draganddrop.asp

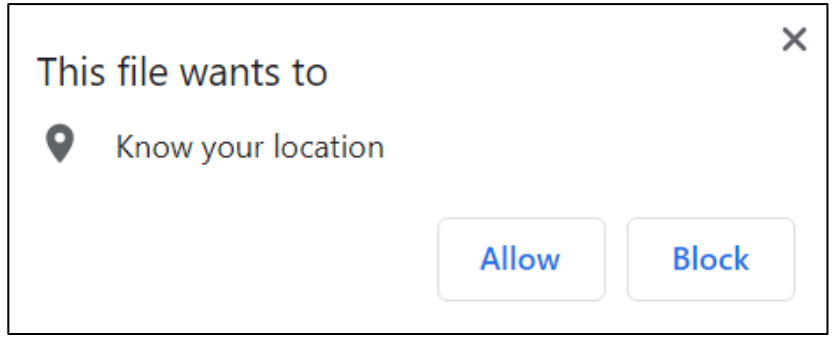
GeoLocation

HTML APIs

1. What is an API?
2. Drag and Drop
3. GeoLocation
4. Storage

Geolocation

- The HTML5 Geolocation API is used to get the geographical position of a user.
- Since this can compromise user privacy, the position is not available unless the user approves it.
- Geolocation is very useful for location-specific information.
  - Examples:
    - Up-to-date local information
    - Showing Points-of-interest near the user
    - Navigation (GPS)



Using Geolocation

- API provides 3 methods that are packed into the navigator.geolocation object
  - getCurrentPosition():
    - Retrieves the device's current location
  - watchPosition():
    - Registers a handler function that will be called automatically each time the position of the device changes, returning the updated location.
  - clearWatch():
    - Stops the watchPosition() method

Finding a User's Coordinates…

Success callback

Error callback

```
function getLocation() {
   var element = document.getElementById("location");
   if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showPosition, showError);
        element.innerHTML = "Getting the position information...";
    } else {
        element.innerHTML = "Geolocation is not supported by this browser.";
    }
}
```

…Finding a User's Coordinates

```
function showPosition(position) {
   var coords = position.coords.latitude + "," + position.coords.longitude;
   document.getElementById("location").innerHTML = "Your location is:" + coords;
}
```

Dealing with Errors

```
function showError(error) {
    var element = document.getElementById("location");
    switch (error.code) {
    case error.PERMISSION_DENIED:
        element.innerHTML = "User denied the request for Geolocation."
        break;
    case error.POSITION_UNAVAILABLE:
        element.innerHTML = "Location information is unavailable."
        break;
    case error.TIMEOUT:
        element.innerHTML = "The request to get user location timed out."
        break;
    case error.UNKNOWN_ERROR:
        element.innerHTML = "An unknown error occurred."
        break;
    }
}
```

Storage

HTML APIs

1. What is an API?
2. Drag and Drop
3. GeoLocation
4. Storage

Reasons to use Client-Side Storage

- Make app work when the user is offline
  - Possibly sync'ing data back once the network is connected again.
- Performance booster
  - Show a large corpus of data as soon as the user clicks on to site, instead of waiting for it to download again
    - should only use it for non-critical data
  - Reduce number of HTTP requests
- Easier programming model
  - with no server infrastructure required

Storage Options in HTML5

- AppCache
- Web Storage API
  - localStorage
  - sessionStorage
- Cookies
- File APIs
- FileSystem APIs
- WebSQL DB
- Indexed DB

What is Web Storage?

- Web storage feature lets you store some information locally on the user's computer
- Data is stored in **key/value** pairs
  - **key** identifies the name of the information (like 'first_name'),
  - **value** is the value associated with that key (say 'Peter')
- Web page can only access data stored by itself
- Persistent on page reloads
- Prefer for storing user preferences

What is Web Storage?

- Similar to cookies:
  - Faster and much better than cookies.
  - Web storage is no more secure than cookies.
  - The data is not included with every server request, but used ONLY when asked for
  - Cookies let you store a small amount of data (nearly 4KB); the web storage allows you to store up to 5MB of data
  - Data size do not affect the website's performance

Browser Support

- Web storage is supported in
  - Internet Explorer 8+
  - Firefox
  - Opera
  - Chrome
  - Safari

The web storage data (both localStorage and sessionStorage) is not shared between different browsers. For example, data stored in Firefox will not be accessible in Google Chrome, Safari, Internet Explorer, or any other browser.

Web Storage Types

- 2 types: which differ in scope and lifetime
  - Local Storage
  - Session Storage

Local Storage

- Uses the *localStorage* object to store data for your entire website on a permanent basis
  - stores data with no expiration date
  - that means the stored local data will be available on the next day, the next week, or the next year unless you remove it.
  - data will not be deleted when the browser is closed

Session storage

- Uses the *sessionStorage* object to store data on a temporary basis, for a single browser window or tab.
- The data disappears when session ends i.e. when the user closes that browser window or tab.

localStorage Object

- localStorage.setItem(key, value)
  - stores the value associated with a key
- localStorage.getItem(key)
  - retrieves the value associated with the key
- localStorage.removeItem(key)
  - remove a particular item
- localStorage.clear()
  - remove the complete storage
- localStorage.key(index)
  - returns name of the key with the specified index
- localStorage.length
  - returns the number of items stored in the browsers Storage Object

localStorage example

```
if (localStorage) {
    localStorage.setItem(‘prefCode', 123);
    localStorage.getItem(‘prefCode'); // value
    localStorage.setItem('favoriteColor', 'yellow’);
    localStorage.length // num of items stored
    localStorage.key(0); // 'prefCode’
    localStorage.removeItem('favoriteColor’);
    localStorage.clear();
}else{
    alert("Sorry, your browser do not support local storage.");
}
```

sessionStorage Object

- sessionStorage
  - Equal to the localStorage object, except that it stores the data for only one session
    - Data is deleted when the user closes the browser window
  - Prefer for storing sensitive data

sessionStorage Example

```
// Check if the sessionStorage object exists
if(sessionStorage) {
        // Store data
        sessionStorage.setItem("last_name", "Parker");
        // Retrieve data
        alert("Hi, " + localStorage.getItem("first_name") + " " +
        sessionStorage.getItem("last_name"));
} else {
        alert("Sorry, your browser do not support session storage.");
}
```

More Info

- https://scriptverse.academy/tutorials/html5-web-storage.html
- https://javascript.info/data-storage
- https://www.w3schools.com/html/html5_webstorage.asp

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr